① Visão geral · o ciclo de uma tela reativa em Angular

Mini Tarefas: os dados entram → o Service guarda o estado → os componentes mostram → o usuário age → o Service atualiza → a tela muda sozinha

ENTRADAS E DADOS

Usuário
clica, digita na busca, marca ✓ e envia o formulário

Servidor
`public/tarefas.json`
a nossa "API" de exemplo

① GET

④ clique · digitação · submit

TarefaService · @Service() · criado com ng g s · o "cérebro" do app

HttpClient · Observable
`http.get<Tarefa[]>()` devolve um **Observable**. `retry` · `catchError` · `finalize` tratam falha e "carregando".

② subscribe → set(lista)

Injeção de Dependência
`@Service()` cria **1 única instância** (singleton). Quem precisa, pede: `inject(TarefaService)`

③ inject()

lê signals ( )

signal · tarefas
`signal<Tarefa[]>([])` é a **fonte única da verdade**. Também guarda `carregando` e `erro`.

⑥

Ações (métodos)
`adicionar · alternar · remover` usando `update(l => [...])`, sempre **imutável**.

⑤ chama métodos

lê tarefas()

avisar(msg)

⑦ recalcula

computed
`total · concluidas · pendentes`: valores derivados que recalculam sozinhos.

NotificacaoService
`Subject` ("rádio ao vivo"): `avisar(msg)` → `mensagem$`

mensagem$

getItem (ao iniciar)

localStorage
memória do navegador: as tarefas sobrevivem ao F5

setItem (salva)

effect
roda quando `tarefas()` muda e **salva no localStorage**.

Regra de ouro
Nunca `push()` direto. Crie um array **novo**: assim signals e OnPush percebem a mudança.

COMPONENTES · o "rosto"

App
`effect` → título da aba `(4) Mini Tarefas`

Resumo
`computed percentual` + `total()` `pendentes()`

Formulario
digita e chama `service.adicionar()`

Lista
`BehaviorSubject` + `combineLatest` + `| async`

CardTarefa · CampoBusca
`input()` · `output()` · `model()`

Toast
`mensagem$ | async` + `switchMap`

Template
`total()` · `@for` · `@if` · `| async` mostram os dados na tela

OnPush
O componente só atualiza quando: um **input** muda, ocorre um **evento**, um **signal lido** muda ou o **async** emite.

⑧ a tela atualiza sozinha: UI = f(estado)

Service / DI · Signals · Componentes · RxJS (Subject) · HttpClient · Navegador

Linha tracejada = leitura única na inicialização

② Services, Injeção de Dependência e Signals

Onde o estado vive (Service), como os componentes chegam até ele (inject) e como a tela reage quando ele muda (signal · computed · effect)

Services e Injeção de Dependência

1 · Como criar e registrar

Gerar
`ng g s services/tarefa-service` cria a classe do service

decorator →

Registrar
`@Service()` (v22)
`@Injectable({ providedIn: 'root' })` (v17)

resultado →

Singleton
1 instância compartilhada pelo app todo

2 · Como o Angular entrega (Injeção de Dependência)

Injector do Angular
guarda a instância única de `TarefaService` e entrega a quem pedir

também usa →

HttpClient
`inject(HttpClient)`
precisa de `provideHttpClient()`

NotificacaoService
`inject(NotificacaoService)`
service usando service

↑ `inject(TarefaService)` : cada componente PEDE e recebe a MESMA instância

App
effect do título

Resumo
totais

Formulario
adicionar

Lista
filtros

3 · Por que não usar `new`?

❌ **new TarefaService()**
cada componente cria uma cópia: o Resumo mostra 5 e a Lista mostra 6 (dados divergem).

✅ **inject(TarefaService)**
todos recebem a mesma instância: adicionou em um lugar, todos veem.

4 · Providers: quem registra o que

app.config.ts
`provideHttpClient()` habilita o `HttpClient` para o app inteiro

sem ele →

NullInjectorError
No provider for HttpClient

💡 **Regra prática:** lógica ou dado usado por mais de um componente vai para um **Service**. Componente só mostra e dispara ações.

Signals: signal · computed · effect

O fluxo de dependência (exemplo real do projeto)

signal · fonte (você escreve)
`tarefas = signal<Tarefa[]>([...5 tarefas])`
troca com `set(valor)` ou `update(fn)`; lê com `tarefas()`

⬇ quem lê `tarefas()` passa a depender dele

computed · derivado (só leitura)
`total = 5`
`concluidas = 1`
`pendentes = 4`
recalcula sozinho; guarda o resultado (cache)

effect · efeito colateral
① `localStorage.setItem(...)`
② `document.title = '(4) Mini Tarefas'`
não devolve valor; roda de novo quando o signal muda

⬇ o template lê com parênteses

Template
`{{ service.total() }}` · `[style.width.%]="percentual()"`
quando o signal muda, só o trecho afetado atualiza

Quando uso cada um?

Preciso de...	Uso	No projeto
guardar um valor que muda	`signal`	`tarefas`, `carregando`, `erro`
calcular a partir de outros	`computed`	`pendentes`, `percentual`
fazer algo **fora** da tela	`effect`	localStorage, título da aba

⚠️ **Imutável:** use `update(l => [...l, nova])`. Um `push()` não avisa ninguém.

⚠️ **Parênteses:** `service.total()` mostra o número. `service.total` mostra a função.

💡 Não use `effect` para calcular valores (use `computed`). O `effect` é para **sincronizar com o mundo de fora** (storage, título, log).

③ Componentes: pai ↔ filho, OnPush e template

Como os componentes conversam entre si (input · output · model) e quando o Angular decide atualizar a tela (OnPush)

input · output · model: a conversa entre pai e filho

① Pai ➜ Filho · `input()` (o pai manda um dado)

Lista (pai)
`<app-card-tarefa [tarefa]="t" />`

dado desce ➜

CardTarefa (filho)
`tarefa = input.required<Tarefa>()`
no template: `tarefa().titulo`

② Filho ➜ Pai · `output()` (o filho avisa que algo aconteceu)

CardTarefa (filho)
`alternar = output<number>()`
no clique: `alternar.emit(tarefa().id)`

evento sobe ➜

Lista (pai)
`(alternar)="alternar($event)"`
o pai decide o que fazer com o `id`

③ Pai ⇄ Filho · `model()` (ida e volta = input + output)

Lista (pai)
`<app-campo-busca [(texto)]="busca" />`
`busca = signal('')`

⇄ two-way ⇄

CampoBusca (filho)
`texto = model('')`
digitou: `texto.set(valor)` e o pai já recebe

④ E depois? A ação chega ao Service e a tela atualiza

Lista.alternar(id) ➜ **service.alternar(id)** `update(...)` ➜ **signal muda** computed e effect recalculam ➜ **Tela** card e resumo atualizam

API	Direção	Como ler / usar	Pense como...
`input()`	pai ➜ filho	`tarefa()` (só leitura)	parâmetro de função
`output()`	filho ➜ pai	`.emit(valor)`	callback / evento
`model()`	pai ⇄ filho	`texto()` e `texto.set(v)`	input + output juntos

💡 O **CardTarefa não conhece o service**: é um componente "burro" (só mostra e avisa), por isso é fácil de reutilizar e testar. Quem conversa com o Service é o pai (`Lista`).

OnPush: quando o Angular atualiza um componente?

Algo aconteceu na aplicação
clique, resposta HTTP, timer, digitação...

⬇

❓ **O componente é OnPush?**

Default (Angular ≤ 21)
verifica **todos** os componentes sempre.
Simples, porém mais lento.

OnPush (padrão no v22)
verifica o componente **só se**:
① um `input` mudou (nova referência)
② evento do próprio template
③ um `signal` lido nele mudou
④ o `async` pipe emitiu

🔑 **Por isso trabalhamos assim:** estado em **signals**, dados **imutáveis** (`update` com array novo) e Observables com **async pipe**. Com isso o OnPush "enxerga" tudo.

⚠️ Alterar o array com `push()` mantém a mesma referência: o OnPush **não percebe** e a tela fica velha.

No Angular 22, **zoneless** (sem zone.js) também é padrão. No projeto escrevemos `ChangeDetectionStrategy.OnPush` no CardTarefa e na Lista para fixar.

Template: control flow moderno

`@if` mostrar condicionalmente
`@if (service.carregando()) { <p>Carregando...</p> }`

`@for` repetir (com `track`)
`@for (t of tarefas; track t.id) { <app-card-tarefa … /> }`
`track t.id` = chave única para o Angular reaproveitar os itens

`@empty` lista vazia
`@empty { <p>Nenhuma tarefa encontrada.</p> }`

`@if (x | async; as v)`: o **async pipe** assina, guarda em `v` e cancela sozinho ao destruir o componente.

④ Observables, Subject, BehaviorSubject e HttpClient

Fluxos de dados ao longo do tempo: como o RxJS entra no projeto e qual "tipo" de fluxo usar em cada situação

Observable: um fluxo de valores no tempo

Produtor
HTTP, evento, timer, `Subject`

→ **.pipe()**

Operadores
`map` · `debounceTime` · `retry` … transformam o fluxo

→ **assina**

Consumidor
`| async` (template) ou `.subscribe()`

→ **Tela**
valor exibido



valores emitidos ao longo do tempo (cada bolinha = um valor)

> 💡 Um Observable é **preguiçoso**: nada acontece até alguém assinar (`subscribe` ou `async`). Ele pode emitir 0, 1 ou vários valores, e pode terminar com **erro** ou **complete**.

Subject × BehaviorSubject

Subject = rádio ao vivo 📻
não guarda valor: quem entra depois **perde** o que já passou.
Uso: eventos (aviso, toast).
No projeto: `NotificacaoService`

BehaviorSubject = placar 🏟️
guarda o **último valor**: quem chega vê o resultado atual na hora.
Uso: estado (categoria, tema).
No projeto: `categoria$` na Lista

	Subject	BehaviorSubject
valor inicial	não tem	obrigatório: `new BehaviorSubject('Todas')`
emitir	`.next(valor)` nos dois	
expor para fora	`.asObservable()` : os outros só leem, ninguém chama `next`	

Exemplo: o aviso verde (Toast)

avisar(msg)
`subject.next(msg)`

→

mensagem$
Observable do Subject

→ **switchMap**

timer(3000)
mostra a msg e some em 3 s

→ **| async**

Toast
aviso verde

> `switchMap` : uma mensagem nova **cancela o timer da anterior**. Por isso o aviso nunca "pisca" nem some antes da hora.

HttpClient: buscar dados e tratar os 3 estados

Antes você usava `fetch` + `await` . O `HttpClient` faz o mesmo, mas devolve um **Observable**: ganhamos `retry` , `catchError` , `delay` , `finalize` e cancelamento "de graça".

1 · carregar()
`carregando.set(true)` · `erro.set('')`

⬇

2 · http.get<Tarefa[]>('tarefas.json')
devolve o Observable (ainda não executou!)

⬇ **.pipe(…)**

3 · delay(800)
atraso de mentira só para ver o "Carregando…"

⬇

4 · retry(2)
falhou? tenta de novo até 2 vezes (veja na aba Network)

⬇

5 · catchError(() => …)
`erro.set('Não foi possível…')` e devolve `EMPTY` (termina sem valor)

⬇

6 · finalize(…)
`carregando.set(false)` : roda em **sucesso OU erro**

⬇ **.subscribe(lista => …)**

7 · tarefas.set(lista)
o signal muda e a tela atualiza

⏳ **Carregando**
`carregando()` = true

✅ **Sucesso**
lista preenchida

❌ **Erro**
`erro()` + botão "Tentar novamente"

⑤ Busca reativa: onde Signals, RxJS e Template se encontram

O pipeline da `Lista`: três fontes de dados viram uma lista filtrada, mostrada no template com async pipe

tarefasFiltradas$ = combineLatest([tarefas, busca, categoria])

Fonte 1 · tarefas
`service.tarefas` (signal)

→ **toObservable**

Observable
emite a lista a cada mudança

Usuário digita
`<app-campo-busca>`

→ **[(texto)]**

signal busca
`busca = signal('')`

→ **toObservable**

pipe de operadores
`debounceTime(300)` espera parar de digitar
`distinctUntilChanged()` ignora repetido
`map(trim · lower)` padroniza
`startWith('')` abre mostrando tudo

→ **último valor de cada**

Escolhe categoria
`<select (change)>`

→ **.next(valor)**

Fonte 3 · categoria$
`new BehaviorSubject('Todas')`
guarda o valor atual (placar)

combineLatest
junta o **ÚLTIMO** valor de cada fonte e emite sempre que **qualquer uma** mudar
só começa quando todas já emitiram uma vez

→

map(([tarefas, texto, cat]) => ...)
`tarefas.filter(...)`
categoria bate E título contém o texto

↓

tarefasFiltradas$
Observable com a lista pronta

↓ **| async**

@for → <app-card-tarefa>
cards na tela

🔍 **Teste:** digite "rx" rápido: a lista só filtra ~300 ms **depois** que você para (sem `debounceTime` filtraria a cada tecla e, com HTTP real, dispararia dezenas de requisições).

❓ **Para pensar:** se remover o `startWith('')`, a lista fica vazia ao abrir, pois o `combineLatest` espera **todas** as fontes emitirem.

A ponte entre Signals e RxJS

signal
`busca()`

→ **`toObservable(signal)`**

Observable
agora aceita `debounceTime`, `combineLatest`...
use quando precisa de tempo e combinação

Observable
`http.get(…)`, `valor$`

→ **`toSignal(obs)`**

signal
lê com `()`, serve em `computed` / `effect`
precisa de `initialValue`

Situação	Prefira	Por quê
Mostrar um Observable no template	`| async`	assina e cancela sozinho; sem `subscribe` manual (evita vazamento de memória)
Usar o valor de um Observable em `computed` / `effect` /TS	`toSignal()`	vira signal: lê com `()` em qualquer lugar
Aplicar tempo/combinação num signal	`toObservable()`	entra no mundo do RxJS (debounce, combineLatest...)

RxJS · Signals · Ação do usuário · Template

⑥ Qual ferramenta uso? Guia de decisão + GitFlow

Pergunte-se "o que eu preciso?" e siga a seta. Depois, como entregar o código com o fluxo colaborativo do GitHub

Dados e lógica

Mais de um componente usa o mesmo dado ou regra?	**Service + inject()** `ng g s` · `@Service()`
Preciso de HTTP?	**HttpClient** `provideHttpClient()` + Observable
Um service usa outro?	**DI entre services** `inject()` no service

Estado na tela

É um valor que muda e a tela mostra?	**signal** `set` · `update`
É calculado a partir de outros valores?	**computed** só leitura, com cache
Preciso agir fora da tela quando mudar?	**effect** storage, título, log

Comunicação entre componentes

O pai precisa mandar um dado ao filho?	**input()** `input.required<T>()`
O filho precisa avisar o pai?	**output()** `.emit(valor)`
Campo que vai e volta (pai ⇄ filho)?	**model()** `[(texto)]`
Quero evitar verificações à toa?	**OnPush** + signals + dados imutáveis

Fluxos com RxJS

É um aviso/evento que não precisa de memória?	**Subject** `next` · `asObservable`
É um estado com valor atual?	**BehaviorSubject** valor inicial
Juntar várias fontes? Esperar parar de digitar?	**combineLatest · debounceTime**
Nova emissão deve cancelar a anterior?	**switchMap**
Mostrar Observable no template? Trocar signal ⇄ Observable?	**| async · toSignal · toObservable**

GitFlow colaborativo: do código ao Pull Request

1. **Partir da develop** — `git checkout develop` `git pull`
2. **Criar a branch** — `git checkout -b feature/mini-tarefas`
3. **Commits pequenos** — `git add .` `git commit -m "feat: ..."`
4. **Enviar** — `git push -u origin feature/mini-tarefas`
5. **Pull Request** — abrir PR para `develop`, pedir revisão da dupla
6. **Merge** — aprovado: entra na `develop` (e depois na `main`)

Onde cada tópico aparece no projeto Mini Tarefas

Service + DI
`tarefa-service.ts` · `inject()` em Resumo, Formulario, Lista, App

signal · computed · effect
TarefaService · `percentual` (Resumo) · título da aba (App)

input · output · model · OnPush
`card-tarefa` · `campo-busca` · `lista`

BehaviorSubject · combineLatest · async
`lista.ts` e `lista.html`

Service / DI · Signals · Componentes · RxJS · HttpClient · Git